

# DW\_DataJobs — SSIS Data Warehouse Project Documentation

## Table of Contents

- 1. [Project Overview](#)
- 2. [Source Data Analysis](#)
- 3. [Data Warehouse Model — Design Choices](#)
- 4. [ETL Pipeline — Step by Step](#)
- 5. [Data Cleaning & Preparation](#)
- 6. [Technical Decisions — Why This, Not That](#)
- 7. [Final Results & Validation](#)

## 1. Project Overview

| Item         | Detail                                                                                                      |
|--------------|-------------------------------------------------------------------------------------------------------------|
| Goal         | Build a star schema data warehouse from raw job posting data using SSIS only (no Python, no external tools) |
| Source       | <code>data_jobs.csv</code> — 785,741 rows, 17 columns                                                       |
| Target       | SQL Server 2022, database <code>DW_DataJobs</code>                                                          |
| Tool         | Visual Studio 2026 with SSIS extension                                                                      |
| Architecture | Star schema + bridge tables for M:N relationships                                                           |
| ETL Style    | SQL-based (Execute SQL Tasks), no Data Flow components                                                      |

## 2. Source Data Analysis

Raw CSV Structure (`data_jobs.csv`)

| Column                         | Sample Value                         | Issues Found                                                |
|--------------------------------|--------------------------------------|-------------------------------------------------------------|
| <code>job_title_short</code>   | <code>Data Analyst</code>            | Clean, used directly as job category                        |
| <code>job_title</code>         | <code>Data Analyst - Remote</code>   | Long strings, up to 500 chars                               |
| <code>job_location</code>      | <code>Boston, MA</code>              | Mixed format: "City, State", "Anywhere", blank, single city |
| <code>job_via</code>           | <code>via LinkedIn</code>            | Always prefixed with "via " — needs stripping               |
| <code>job_schedule_type</code> | <code>Full-time and Part-time</code> | Multi-valued: "X", "X and Y", "X, Y, and Z"                 |

| Column                             | Sample Value                                               | Issues Found                                                                               |
|------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>job_work_from_home</code>    | <code>True / False</code>                                  | String booleans, not BIT                                                                   |
| <code>search_location</code>       | <code>Texas, United States</code>                          | Informational, kept as-is                                                                  |
| <code>job_posted_date</code>       | <code>2023-01-17 06:14:28</code>                           | String datetime, needs TRY_CAST                                                            |
| <code>job_no_degree_mention</code> | <code>True / False</code>                                  | String booleans                                                                            |
| <code>job_health_insurance</code>  | <code>True / False</code>                                  | String booleans                                                                            |
| <code>job_country</code>           | <code>United States</code>                                 | Some blanks/nulls                                                                          |
| <code>salary_rate</code>           | <code>year / hour</code>                                   | Null for ~96% of rows                                                                      |
| <code>salary_year_avg</code>       | <code>65000.0</code>                                       | Null for most rows, string format                                                          |
| <code>salary_hour_avg</code>       | <code>30.5</code>                                          | Null for most rows, string format                                                          |
| <code>company_name</code>          | <code>Google</code>                                        | Some with special unicode chars (caused encoding errors)                                   |
| <code>job_skills</code>            | <code>['python', 'sql']</code>                             | Python list syntax (single quotes), not used — redundant with <code>job_type_skills</code> |
| <code>job_type_skills</code>       | <code>{'programming': ['python'], 'cloud': ['aws']}</code> | Python dict syntax — richer data with categories                                           |

## Key Data Quality Issues

1. **No primary key** — rows have no unique identifier, need to generate `job_id` via `ROW_NUMBER()`
2. **Python data structures** — `job_type_skills` contains Python dict literals with single quotes, not valid JSON
3. **Multi-valued fields** — `job_schedule_type` has 3 different patterns for multiple values
4. **String booleans** — `True/False` instead of 1/0
5. **Encoding** — Some `company_name` values contain characters that fail with UTF-8 CODEPAGE in BULK INSERT
6. **Sparse salary data** — Only ~4% of rows (32,665) have salary information

## 3. Data Warehouse Model — Design Choices

### Why Star Schema (Not Snowflake)?

| Criteria                 | Star                        | Snowflake                 |
|--------------------------|-----------------------------|---------------------------|
| <b>Query performance</b> | Fewer joins = faster        | More joins = slower       |
| <b>Simplicity</b>        | Easy to understand          | More complex              |
| <b>Dimension depth</b>   | All dims are flat (1 level) | Would need sub-dimensions |

| Criteria        | Star                                                 | Snowflake                 |
|-----------------|------------------------------------------------------|---------------------------|
| <b>Our data</b> | No hierarchical dims that benefit from normalization | Overkill for this dataset |

**Decision: Star Schema** — All 8 dimensions are single-level (no company→industry→sector hierarchy exists in the data). Snowflake would add joins with zero benefit.

## Why Bridge Tables (Not Junk Dimensions)?

The data has two many-to-many relationships:

- 1 job → many skills (avg ~4.7 skills per job)
- 1 job → many schedule types (some jobs are "Full-time and Part-time")

| Approach                       | Pros                                       | Cons                                                                        |
|--------------------------------|--------------------------------------------|-----------------------------------------------------------------------------|
| <b>Bridge table</b>            | Clean M:N, easy to query, standard pattern | Extra join                                                                  |
| <b>Junk dimension</b>          | Fewer tables                               | Combinatorial explosion (257 skills × combinations = millions of junk rows) |
| <b>Comma-separated in fact</b> | Simple                                     | Cannot filter, aggregate, or join on individual values                      |

**Decision: Bridge tables** — `bridge.job_skill` and `bridge.job_schedule` are the standard star schema solution for M:N. A junk dimension would be impractical with 257 distinct skills.

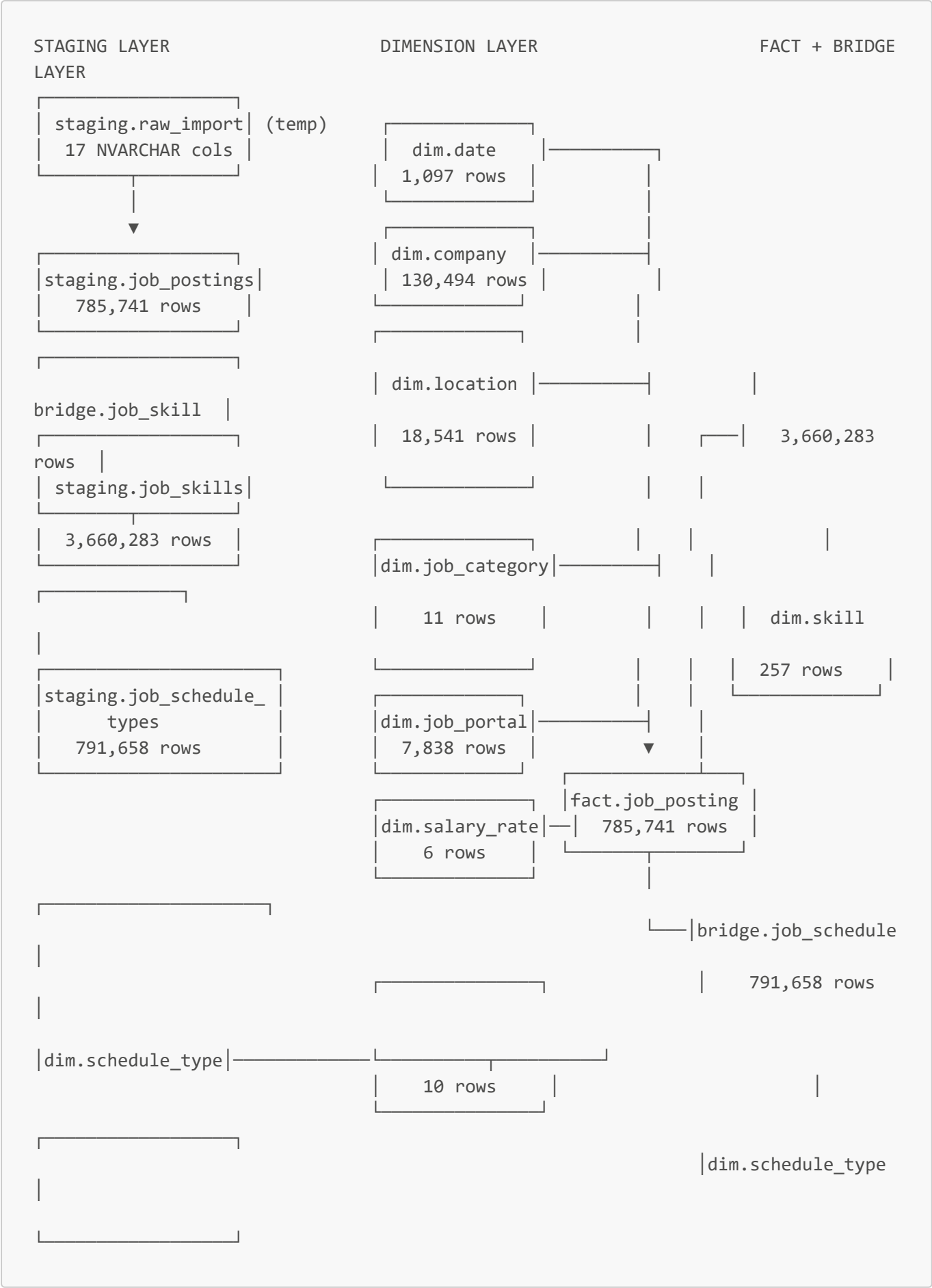
## Why Unknown Members (-1 Key)?

When a job posting has no portal, no salary rate, or no company match, the fact table still needs a valid FK value.

| Approach                   | Pros                                                                | Cons                                                           |
|----------------------------|---------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Unknown member (-1)</b> | FK integrity preserved, no NULLs in FK columns, queries don't break | Extra row per dim                                              |
| <b>NULL FK</b>             | Simple                                                              | Breaks FK constraints, requires ISNULL everywhere in queries   |
| <b>Skip the row</b>        | Clean data                                                          | Loses data — 752,674 jobs with no salary rate would be dropped |

**Decision: Unknown member rows** — Every dimension has a row with key = -1 and name = 'Unknown'. The fact table uses `ISNULL(dim_key, -1)` during load. This is the Kimball best practice.

## Schema Diagram



#### 4. ETL Pipeline — Step by Step

## Package 00: Create Database (`00_Create_Database.dtsx`)

**Purpose:** Make the project fully self-contained — creates the entire database schema from scratch.

| Task                                    | What It Does                                                                                                                                                                                                                                   |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SQL_Reset_Schema</code>           | Drops ALL foreign keys dynamically ( <code>sys.foreign_keys</code> loop), then <code>DROP TABLE IF EXISTS</code> for all 16 tables in FK-safe order (bridge → fact → dim → staging → etl), creates 5 schemas (staging, dim, fact, bridge, etl) |
| <code>SQL_Create_All_Tables</code>      | Creates 3 staging tables, 8 dimension tables with PKs + UNIQUE constraints, 1 fact table, 2 bridge tables, 10 FK constraints, 1 ETL audit table with computed <code>duration_seconds</code> column                                             |
| <code>SQL_Insert_Unknown_Members</code> | Inserts -1 key unknown member row in all 8 dimensions using <code>SET IDENTITY_INSERT ON/OFF</code>                                                                                                                                            |

**Flow:** Reset → Create → Unknown Members (sequential)

## Package 01: Load Staging (`01_Load_Staging.dtsx`)

**Purpose:** Read raw CSV, land it in a staging table, then transform/clean/split into 3 normalized staging tables.

| Task                                 | What It Does                                                                                                                                                                                                                           |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SQL_Prepare_Staging</code>     | Creates <code>staging.raw_import</code> (17 NVARCHAR columns), drops all NC indexes on staging, truncates all 3 staging tables                                                                                                         |
| <code>SQL_BulkLoad_Raw</code>        | <code>BULK INSERT</code> from <code>data_jobs.csv</code> with <code>FORMAT='CSV', TABLOCK</code> (minimal logging), <code>BATCHSIZE=100000</code>                                                                                      |
| <code>SQL_Transform_Jobs</code>      | Transforms <code>raw_import</code> → <code>staging.job_postings</code> : generates <code>job_id</code> via <code>ROW_NUMBER()</code> , parses location, strips "via " prefix, converts booleans, <code>TRY_CASTs</code> dates/decimals |
| <code>SQL_Transform_Skills</code>    | (Parallel) Parses Python dict → JSON → rows in <code>staging.job_skills</code> : <code>REPLACE(CHAR(39), CHAR(34)) + OPENJSON</code> to extract skill name + category                                                                  |
| <code>SQL_Transform_Schedules</code> | (Parallel) Splits multi-valued schedule types: <code>REPLACE(' and ', ', ')</code> + <code>STRING_SPLIT</code> → rows in <code>staging.job_schedule_types</code>                                                                       |
| <code>SQL_Rebuild_Staging</code>     | Drops <code>raw_import</code> , creates covering NC indexes on staging tables, <code>UPDATE STATISTICS SAMPLE 50%</code>                                                                                                               |
| <code>SQL_Validate_Staging</code>    | <code>RAISERROR</code> if any of the 3 staging tables is empty                                                                                                                                                                         |

**Flow:** Prepare → BulkLoad → Transform\_Jobs → [Skills || Schedules] (parallel) → Rebuild → Validate

## Package 02: Populate Dimensions (`02_Populate_Dimensions.dtsx`)

**Purpose:** Fill all 8 dimension tables from staging data. Idempotent (safe to re-run).

| Task                 | What It Does                                                                                                                                                                                                                |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQL_Dim_Date         | Recursive CTE from 2023-01-01 to 2025-12-31 (MAXRECURSION 1200).<br>Columns: date_key (YYYYMMDD), full_date, day_of_month, day_of_week, day_name, week_of_year (ISO_WEEK), month_num, month_name, quarter, year, is_weekend |
| SQL_Dim_Company      | INSERT ... WHERE NOT EXISTS from staging.job_postings                                                                                                                                                                       |
| SQL_Dim_Location     | INSERT ... GROUP BY ... HAVING NOT EXISTS for deduplication (same raw_location + country = same row)                                                                                                                        |
| SQL_Dim_JobCategory  | From job_title_short — the 11 distinct job categories                                                                                                                                                                       |
| SQL_Dim_JobPortal    | From job_portal after "via " stripping                                                                                                                                                                                      |
| SQL_Dim_ScheduleType | From staging.job_schedule_types distinct values                                                                                                                                                                             |
| SQL_Dim_Skill        | From staging.job_skills — 257 unique (skill_name, skill_category) pairs                                                                                                                                                     |
| SQL_Dim_SalaryRate   | From distinct salary_rate values (year, hour, etc.)                                                                                                                                                                         |
| SQL_Validate_Dims    | Checks all 8 dims have rows with key > 0                                                                                                                                                                                    |

**Flow:** Date → [Company || Location || Category || Portal || Schedule || Skill || SalaryRate] (7 parallel) → Validate

### Package 03: Populate Facts (03\_Populate\_Facts.dtsx)

**Purpose:** Load the central fact table and both bridge tables.

| Task                   | What It Does                                                                                                                                                                                                      |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQL_Prepate_Targets    | <b>Drops</b> FK_bridge_skill_job and FK_bridge_sched_job (required — TRUNCATE fails on FK-referenced tables even with NOCHECK), NOCHECK remaining constraints, TRUNCATE all 3 tables, dynamically drop NC indexes |
| SQL_Fact_JobPosting    | INSERT WITH (TABLOCK) — LEFT JOIN to all 6 dimensions, ISNULL(key, -1) for unknown member mapping, MIN(location_key) subquery for location dedup, computes has_salary_info flag                                   |
| SQL_Bridge_Skill       | (Parallel) INSERT with TABLOCK — joins staging.job_skills → dim.skill → fact.job_posting                                                                                                                          |
| SQL_Bridge_Schedule    | (Parallel) INSERT with TABLOCK — joins staging.job_schedule_types → dim.schedule_type → fact.job_posting                                                                                                          |
| SQL_Enable_FK_Validate | <b>Recreates</b> the 2 dropped bridge→fact FKs, WITH CHECK CHECK CONSTRAINT ALL on all tables (marks FKs as trusted), RAISERROR if any table is empty                                                             |

**Flow:** Prepare → Fact → [Bridge\_Skill || Bridge\_Schedule] (parallel) → Enable\_FK + Validate

## Package 04: Finalize (04\_Finalize.dtsx)

**Purpose:** Build performance indexes, update statistics, run final FK orphan audit.

| Task                 | What It Does                                                                                                                |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| SQL_Indexes_Fact     | (Parallel) 7 NC indexes on fact.job_posting — all with <code>DATA_COMPRESSION=PAGE, FILLFACTOR=90, SORT_IN_TEMPDB=ON</code> |
| SQL_Indexes_Bridge   | (Parallel) 4 NC indexes on bridge tables                                                                                    |
| SQL_Indexes_Dim      | (Parallel) 4 NC indexes on dimension tables                                                                                 |
| SQL_Update_Stats     | <code>UPDATE STATISTICS</code> with <code>FULLSCAN</code> on fact + bridge, <code>SAMPLE 50 PERCENT</code> on dims          |
| SQL_Final_Validation | Checks all 10 FK relationships for orphan rows (skipping -1 unknown keys), <code>RAISERROR</code> on any violation          |

**Flow:** [Fact\_Idex || Bridge\_Idex || Dim\_Idex] (3 parallel) → Stats → Validation

## Package Master: Orchestrator (Master.dtsx)

**Purpose:** Run all packages in sequence with ETL audit logging.

| Task                       | What It Does                                                                  |
|----------------------------|-------------------------------------------------------------------------------|
| SQL_ETL_Start              | Inserts a row in <code>etl.run_log</code> with status = 'Running'             |
| EPT_00_Create_Database     | ExecutePackageTask → 00_Create_Database.dtsx                                  |
| EPT_01_Load_Staging        | ExecutePackageTask → 01_Load_Staging.dtsx                                     |
| EPT_02_Populate_Dimensions | ExecutePackageTask → 02_Populate_Dimensions.dtsx                              |
| EPT_03_Populate_Facts      | ExecutePackageTask → 03_Populate_Facts.dtsx                                   |
| EPT_04_Finalize            | ExecutePackageTask → 04_Finalize.dtsx                                         |
| SQL_ETL_Complete           | Updates <code>etl.run_log</code> with end time, row counts, dim count summary |

**Flow:** ETL\_Start → 00 → 01 → 02 → 03 → 04 → ETL\_Complete (all sequential)

## 5. Data Cleaning & Preparation

### 5.1 Landing Phase (Raw → raw\_import)

The raw CSV is loaded into `staging.raw_import` with **all 17 columns as NVARCHAR** — no type conversion at this stage. This absorbs any data quality issues without failure.

**BULK INSERT configuration:**

- `FORMAT = 'CSV'` — handles quoted fields (critical: `job_type_skills` contains commas inside Python dicts)
- `FIRSTROW = 2` — skip header
- `TABLOCK` — table lock for minimal logging (bulk-logged recovery)
- `BATCHSIZE = 100000` — commit every 100K rows (balance between speed and transaction log)
- No `CODEPAGE` — removed after discovering row 533,102 had a `company_name` with characters that failed UTF-8 codepage conversion

## 5.2 Job Postings Transform

### Unique ID generation:

```
ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS job_id
```

The CSV has no primary key. `ROW_NUMBER()` assigns a sequential ID. `ORDER BY (SELECT NULL)` means insertion order — deterministic within the same BULK INSERT.

### Location parsing:

```
-- "Boston, MA" → city="Boston", state_region="MA"
-- "Anywhere" → city=NULL, state_region=NULL (work from home)
-- "London" → city="London", state_region=NULL (no comma)
-- "" or NULL → city=NULL, state_region=NULL
CASE WHEN CHARINDEX(',', job_location) > 0
      THEN LEFT(job_location, CHARINDEX(',', job_location) - 1)
      ELSE job_location END AS city
```

### Portal name cleaning:

```
-- "via LinkedIn" → "LinkedIn"
CASE WHEN job_via LIKE 'via %'
      THEN SUBSTRING(job_via, 5, LEN(job_via))
      ELSE job_via END
```

### Boolean conversion:

```
-- "True"/"False" → 1/0
CASE WHEN UPPER(job_work_from_home) = 'TRUE' THEN 1 ELSE 0 END
```

### Safe type casting:

```
-- String → DATETIME2 (returns NULL on bad data instead of failing)
TRY_CAST(job_posted_date AS DATETIME2)
```

```
-- String → DECIMAL (returns NULL on bad data)
TRY_CAST(salary_year_avg AS DECIMAL(12,2))
```

### Company name truncation:

```
LEFT(LTRIM(RTRIM(company_name)), 300)
```

Raw data had company names up to 1000 chars; dimension column is 300. `LEFT()` prevents truncation errors.

## 5.3 Skills Transform (Python Dict → Relational Rows)

The `job_type_skills` column contains Python dictionary literals:

```
{'programming': ['python', 'sql'], 'cloud': ['aws', 'azure']}
```

### Step 1 — Python single quotes → JSON double quotes:

```
REPLACE(job_type_skills, CHAR(39), CHAR(34))
-- Result: {"programming": ["python", "sql"], "cloud": ["aws", "azure"]}
```

### Step 2 — Extract categories with OPENJSON (level 1):

```
CROSS APPLY OPENJSON(jdoc) cat
-- key = "programming", value = '["python", "sql"]'
-- key = "cloud", value = '["aws", "azure"]'
```

### Step 3 — Extract individual skills with OPENJSON (level 2):

```
CROSS APPLY OPENJSON(cat.skill_array) sk
-- value = "python", "sql", "aws", "azure"
```

**Critical fix — job\_id consistency:** The `ROW_NUMBER()` must be computed over ALL rows (not just non-null ones) to match the `job_id` in `staging.job_postings`:

```
-- WRONG: ROW_NUMBER() after WHERE filter → mismatched IDs
WITH numbered AS (
    SELECT ROW_NUMBER()... FROM raw_import
    WHERE job_type_skills IS NOT NULL -- ❌ Numbers only filtered rows
)

-- CORRECT: ROW_NUMBER() before filter → IDs match job_postings
```

```
WITH numbered AS (
    SELECT ROW_NUMBER()... FROM raw_import -- ✅ Numbers ALL rows
),
filtered AS (
    SELECT * FROM numbered WHERE job_type_skills IS NOT NULL
)
```

## 5.4 Schedule Type Transform (Multi-Value Split)

The `job_schedule_type` column has 3 patterns:

- Single: `Full-time`
- Two with "and": `Full-time and Part-time`
- Three with comma+and: `Full-time, Part-time, and Contractor`

**Solution — Normalize then split:**

```
-- Step 1: Replace " and " with comma
REPLACE(job_schedule_type, ' and ', ',')
-- "Full-time, Part-time, , Contractor" (extra comma is harmless)

-- Step 2: Split on comma
CROSS APPLY STRING_SPLIT(schedule_csv, ',')

-- Step 3: Trim + filter empty
WHERE NULLIF(LTRIM(RTRIM(value)), '') IS NOT NULL
```

## 5.5 Dimension Deduplication

**Location dedup challenge:** Multiple staging rows may have the same `raw_location` + `country` but different `city/state` parsing. Solution:

```
INSERT INTO dim.location (raw_location, city, state_region, country)
SELECT DISTINCT raw_location, city, state_region, country
FROM staging.job_postings
GROUP BY raw_location, city, state_region, country
HAVING NOT EXISTS (
    SELECT 1 FROM dim.location l
    WHERE ISNULL(l.raw_location, '') = ISNULL(s.raw_location, '')
)
```

**All other dims** use `UNIQUE` constraints + `NOT EXISTS` for idempotent inserts.

## 6. Technical Decisions — Why This, Not That

Why Execute SQL Tasks (Not Data Flow)?

| Execute SQL Task               | Data Flow Task                                      |
|--------------------------------|-----------------------------------------------------|
| All logic in T-SQL on server   | Data moves through SSIS engine in memory            |
| 785K rows processed in seconds | Would need to stream 785K rows through pipeline     |
| Server-side BULK INSERT        | Flat File Source → OLE DB Destination               |
| OPENJSON, STRING_SPLIT native  | Would need Script Component for Python dict parsing |
| Parallel operations via T-SQL  | Needs multiple data flows                           |

**Decision:** Execute SQL Tasks. The data transformations (JSON parsing, string splitting, location parsing) are all T-SQL native. Moving data through SSIS Data Flow would be slower and more complex for this use case.

Why BULK INSERT (Not Flat File Source)?

| BULK INSERT                                 | Flat File Source                  |
|---------------------------------------------|-----------------------------------|
| Server-side, TABLOCK = minimal logging      | Client-side, row-by-row           |
| Handles CSV quoted fields with FORMAT='CSV' | Needs text qualifier config       |
| 785K rows in ~2 minutes                     | Would take 10+ minutes            |
| Simple single SQL statement                 | Needs Data Flow + column mappings |

**Decision:** BULK INSERT with `FORMAT='CSV'` — handles the quoted Python dicts that contain commas.

Why MSOLEDBSQL (Not SQLNCLI11)?

`SQLNCLI11.1` (SQL Server Native Client 11) is deprecated and not installed on new machines. `MSOLEDBSQL` (Microsoft OLE DB Driver for SQL Server) is the modern replacement, ships with SQL Server 2022.

Why Drop FKs Before TRUNCATE (Not DELETE)?

SQL Server **does not allow** `TRUNCATE TABLE` on a table referenced by a foreign key — even with `NOCHECK CONSTRAINT`. This is a hard engine limitation.

| Approach                         | Speed                       | Works with FKs?           |
|----------------------------------|-----------------------------|---------------------------|
| <code>TRUNCATE TABLE</code>      | Instant (deallocates pages) | ❌ Not if referenced by FK |
| <code>DELETE FROM</code>         | Slow (row-by-row logging)   | ✅ With NOCHECK            |
| Drop FK → TRUNCATE → Recreate FK | Instant + small overhead    | ✅ Our approach            |

**Decision:** Drop the 2 bridge→fact FKs, TRUNCATE, then recreate them. Best of both worlds.

Why date\_key = YYYYMMDD Integer (Not IDENTITY)?

| YYYYMMDD integer                                     | IDENTITY                       |
|------------------------------------------------------|--------------------------------|
| Human-readable: <code>20230601</code> = June 1, 2023 | <code>437</code> means nothing |

| YYYYMMDD integer                             | IDENTITY                      |
|----------------------------------------------|-------------------------------|
| Can partition by range                       | Range partition harder        |
| WHERE date_key BETWEEN 20230101 AND 20231231 | Need join for any date filter |
| Standard Kimball pattern                     | Non-standard                  |

Why PAGE Compression on Indexes?

Fact + bridge tables have millions of rows with repetitive FK values. `DATA_COMPRESSION = PAGE` reduces storage by 40-60% and improves I/O-bound query performance. `FILLFACTOR = 90` leaves 10% free space for future inserts without page splits.

## 7. Final Results & Validation

Row Counts

| Table                      | Rows                           |
|----------------------------|--------------------------------|
| staging.job_postings       | 785,741                        |
| staging.job_skills         | 3,660,283                      |
| staging.job_schedule_types | 791,658                        |
| dim.date                   | 1,097 (1,096 days + 1 unknown) |
| dim.company                | 130,494                        |
| dim.location               | 18,541                         |
| dim.job_category           | 11                             |
| dim.job_portal             | 7,838                          |
| dim.schedule_type          | 10                             |
| dim.skill                  | 257                            |
| dim.salary_rate            | 6                              |
| <b>fact.job_posting</b>    | <b>785,741</b>                 |
| <b>bridge.job_skill</b>    | <b>3,660,283</b>               |
| <b>bridge.job_schedule</b> | <b>791,658</b>                 |

Integrity Checks — All Passed

| FK Check           | Orphan Rows |
|--------------------|-------------|
| fact → dim.date    | 0           |
| fact → dim.company | 0           |

| FK Check                                | Orphan Rows |
|-----------------------------------------|-------------|
| fact → dim.location                     | 0           |
| fact → dim.job_category                 | 0           |
| fact → dim.job_portal                   | 0           |
| fact → dim.salary_rate                  | 0           |
| bridge.job_skill → fact                 | 0           |
| bridge.job_skill → dim.skill            | 0           |
| bridge.job_schedule → fact              | 0           |
| bridge.job_schedule → dim.schedule_type | 0           |

### FK Trust Status — All Trusted

All 10 foreign keys have `is_not_trusted = 0` (trusted), meaning the SQL Server query optimizer can use them for query plan optimization.

### Unknown Member Usage

| Dimension        | Unknown (-1) Count | Meaning                                    |
|------------------|--------------------|--------------------------------------------|
| date_key         | 0                  | All jobs have valid dates                  |
| company_key      | 4                  | 4 jobs with unmatched company              |
| location_key     | 0                  | All locations resolved                     |
| job_category_key | 0                  | All categories matched                     |
| portal_key       | 8                  | 8 jobs with unmatched portal               |
| salary_rate_key  | 752,674            | Expected — 96% of jobs have no salary info |

### Bugs Found & Fixed During Development

| Bug                                                              | Impact                                              | Fix                                                  |
|------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------|
| <code>CODEPAGE='65001'</code> in BULK INSERT                     | Row 533,102 failed on special chars in company_name | Removed CODEPAGE parameter                           |
| <code>company_name NVARCHAR(300)</code> in raw_import            | Truncation errors                                   | Widened to NVARCHAR(1000)                            |
| <code>ROW_NUMBER()</code> after WHERE filter in skills/schedules | job_id mismatch between staging tables              | Number ALL rows first, filter after                  |
| <code>SQLNCLI11.1</code> provider                                | Not installed on machine                            | Switched to <code>MSOLEDBSQL</code>                  |
| <code>TRUNCATE</code> on FK-referenced table                     | SQL Server blocks it even with NOCHECK              | Drop bridge→fact FKs before truncate, recreate after |

| Bug                            | Impact                                                         | Fix                                         |
|--------------------------------|----------------------------------------------------------------|---------------------------------------------|
| dim.date column names mismatch | Old schema had month, day but SQL used month_num, day_of_month | Rebuilt schema with consistent column names |